Database Normalisation – Explanation

INF10025 Data Management and Analytics · Study guide

1. What is normalisation?

Normalisation is the process of organising the tables of a relational database so that each fact is stored **once, in one place**. It works by decomposing a large, redundant table into smaller, well-structured tables that are linked by primary keys and foreign keys. The decomposition must be **lossless**: joining the smaller tables back together must reproduce the original data.

2. Why do we normalise?

Without normalisation, repeated data causes three kinds of **anomalies**:

Anomaly	Meaning	Example (CLIENT_MEMBERSHIP table)
Insertion	Cannot add a fact without adding an unrelated one	Cannot record a new swimming centre until at least one client joins it.
Update	Same fact stored many times; changing one copy leaves inconsistency	Renaming a centre requires editing every membership row.
Deletion	Deleting one fact accidentally removes another	Deleting the last member of a centre also erases the centre's details.

3. Key ideas

Functional dependency (FD): A → B means that each value of A determines exactly one value of B (e.g. ClientID → ClientName). **Candidate key:** a minimal set of attributes that determines all other attributes. **Partial dependency:** a non-key attribute depends on only part of a composite key. **Transitive dependency:** a non-key attribute depends on another non-key attribute (A → B → C).

4. The normal forms

Form	Rule	Removes
1NF	Every cell holds one atomic value; no repeating groups or multi-valued columns; each row is unique (has a primary key).	Repeating groups
2NF	In 1NF, and every non-key attribute depends on the **whole** primary key (matters only for composite keys).	Partial dependencies
3NF	In 2NF, and no non-key attribute depends on another non-key attribute: non-key attributes depend on "the key, the whole key, and nothing but the key".	Transitive dependencies
BCNF	In 3NF, and for every FD X → Y, X is a candidate key (a stricter version of 3NF).	Remaining anomalies from overlapping keys

5. Worked example (Pete's Pools style)

Un-normalised data – one row per client, with several centres in one cell:

ClientID	ClientName	Centres joined (CentreName : JoinDate)
C001	An Nguyen	Docklands Wave : 2019-05-22; Northside Aqua : 2021-03-10
C002	Binh Tran	Docklands Wave : 2020-01-15

Step 1 – 1NF. Split the repeating group so each row holds one value per cell. Primary key becomes (ClientID, CentreName).

ClientID	ClientName	CentreName	JoinDate
C001	An Nguyen	Docklands Wave	2019-05-22
C001	An Nguyen	Northside Aqua	2021-03-10
C002	Binh Tran	Docklands Wave	2020-01-15

Step 2 – 2NF. ClientName depends only on ClientID (part of the key) – a partial dependency. Move it into its own table. CentreName also has its own attributes (EstablishmentDate, Description), so a centre table is created too.

Step 3 – 3NF. No transitive dependencies remain. Final design:

Table	Primary key	Other attributes / foreign keys
CLIENT	ClientID	ClientName
SWIMMINGCENTRE	CentreName	EstablishmentDate, Description
MEMBERSHIP	(CentreName, ClientID)	JoinDate; CentreName → SWIMMINGCENTRE, ClientID → CLIENT

This is exactly the structure of Pass 5a–5c: the many-to-many relationship between centres and clients is resolved by the weak (intersection) entity MEMBERSHIP, so every fact is stored once.

6. Trade-offs and exam tips

Normalisation reduces redundancy and protects integrity, but more tables mean more joins; in reporting or data-warehouse systems designers sometimes **denormalise** on purpose for speed. Tips: (1) always identify the primary key first; (2) list the functional dependencies; (3) check 2NF only when the key is composite; (4) after splitting, verify with the question "can I rebuild the original table with a join?".